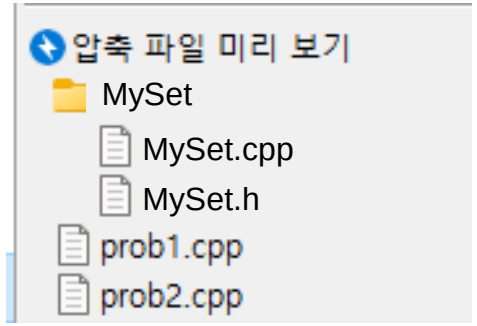
포인터 실습 과제

- 제출 방식 : 10/24(토) 23:00까지 제출
  - {prob1.cpp, prob2.cpp} MySet\{MySet.h, MySet.cpp}, MyMatrix\{MyMatrix.cpp, MyMatrix.h}로 제출
    - 1~2번은 main 하나로만 구성해도 됨.
    - 3번 4번은 class MySet과 class MyMatrix을 위한 .cpp와 .h만 제출
      - default 생성자, 복사 생성자 구현
      - MySet::operator + (const MySet &), MySet::operator &(const MySet &), MySet::operator -(const MySet &), MySet::operator =(const MySet &), MyMatrix::operator + (const MyMatrix &), MyMatrix::operator *(const MyMatrix &), MyMatrixt::operator =(const MyMatrix &)는 멤버 함수로 구현
      - 입출력 operator <<, operator >> 일반 함수로 구현. friend 사용 금지
  - 학번.zip으로 압축하여 제출 (뒤에 자동생성되는 숫자 붙어도 상관없음)



문제 1 : 포인터와 배열의 상호 관계

- 다음 프로그램의 빈 곳을 채워 아래와 같이 구동되게 하시오.

```
int main()
{
    const int MAX = 5;
    int arr[MAX] = { 1,2,3,4,5 };
    for (int* p = ? ? ? ? ? ; p < ? ? ? ? ; p++)
        cout << ? ? ? ? << " ";    // 이 줄에서 arr 사용하지 않고 p만 사용
    cout << endl;
    return 0;
}
```

```
1 2 3 4 5
```

문제 2 : 포인터와 배열 상관관계

- 다음은 컴파일이 제대로 되지 않거나, 정상적으로 동작하지 않는다.
  문제가 되는 부분을 주석처리하고, 동작하는 않는 이유를 주석에 쓰시오.
  - 필요하면 반드시 delete [] 추가하시오.
- 예시의 "원하는 출력"처럼 동작이 되도록 수정하시오.

```
int main()
{
    const int MAX = 5;
    int arr1[MAX] = { 1,2,3,4,5 }, arr2[MAX], * p;

    arr2 = arr1; // arr2가 arr1의 복사본이 되도록 함수 myCopy(int *, int *, size) 작성
    p = arr1;    // p는 arr1의 복사본이 되도록, 동적 할당 후 myCopy() 호출

    p[0] = 99;   // 위의 수정이 제대로 되면 arr2은 안 바뀌면서 p[0]만 수정됨
    print1DArr(arr1, MAX); // print1DArr(int *, int)를 추가 구현할 것
    print1DArr(arr2, MAX);
    print1DArr(p, MAX);

    return  0;
}
```

원하는 출력 :
1 2 3 4 5
1 2 3 4 5
99 2 3 4 5

잘못된 출력:
99 2 3 4 5
1 2 3 4 5
99 2 3 4 5

문제 3 : 클래스의 멤버인 동적 배열

- 자연수 집합을 다루는 아래 프로그램에서 어떤 개수의 정수가 입력되더라도 정상적으로 동작하도록 프로그램을 완성하시오.
  - MySet default 생성자는 size = 0, capacity=4, resize()를 반드시 구현하시오.
    - 대입연산자에서 깊은 복사를 정확하게 구현 / 기존 동적 배열 반드시 delete / MySet의 파괴자를 정확하게 구현 / 함수 선언에서 const 정확하게 지정
  - operator overloading 구현에 유의
    - "집합"은 같은 원소 포함할 수 없으며, 순서가 중요하지 않은 자료형임에 유의
    - 합집합과 교집합, 차집합의 최대 크기를 미리 구하는 방식은 메모리 효율이 떨어지고 포인터 학습에 부적절하므로 resize() 기반으로 구현.
    - 정렬알고리즘을 활용하는 경우 전체적인 속도 효율에 반드시 신경 쓸 것

```
2 3 1 2 4 -1
5 4 3 1 7 9 -1
Union : {2, 3, 1, 4, 5, 7, 9}
Intersection : {3, 1, 4}
Difference : {2} or {5, 7, 9}
```

```
2 3 1 2 4 -1
5 4 3 1 7 9 -1
Union : {1, 2, 3, 4, 5, 7, 9}
Intersection : {1, 3, 4}
Difference : {2} or {5, 7, 9}
```

```
MySet s1, s2, resultSet;

cin >> s1 >> s2;     // 음수가 들어올 때까지 입력
resultSet = s1 + s2;  // 합집합
cout << "Union : " << resultSet << endl;

resultSet = s1 & s2; // 교집합
cout << "Intersection : " << resultSet << endl;

resultSet = s1 - s2;   // 차집합
cout << "Difference : " << resultSet << " or " ;
resultSet = s2 – s1;
cout << resultSet << endl;
```

문제4 : 2차원 동적 배열

- 아래 코드가 오른쪽의 결과를 출력하게 하는, 행렬을 위한 class MyMatrix를 구현하시오.
  - 2차원 동적배열을 사용하시오.
  - 생성자와 파괴자, 각종 연산자와 함수를 정확하게 구현하시오.
  - 연산에 맞지 않는 크기의 행렬이 들어오는 경우, 4번과 6번처럼 -999만 저장된 1x1 행렬 반환

```
const int MAX_CNT = 8 ;
MyMatrix matArr[MAX_CNT] ;
matArr[0] = MyMatrix(2,2) ; // 행렬 크기 초기화
matArr[1] = MyMatrix(2,2) ;
matArr[2] = MyMatrix(2,3) ;

int arr0[] = {1,2,3,4}, arr1[] = {1, -1, 0, 2} ;
int arr2[] = {1,2,3,3,2,1} ; // 배열 내용 초기화
matArr[0].init(arr0) ; // arr0으로 시작하는 배열에
matArr[1].init(arr1) ; // 적절한 개수의 값이 있다고
matArr[2].init(arr2) ; // 가정하고 행렬값 행우선 초기화

matArr[3] = matArr[0] + matArr[1] ;
matArr[4] = matArr[0] + matArr[2] ;
matArr[5] = matArr[0] * matArr[1] ;
matArr[6] = matArr[2] * matArr[0] ;
matArr[7] = matArr[0] * matArr[2] ;

for (int i = 0 ; i < MAX_CNT ; i++) { // 출력
    cout << "-- Matirx " << i << " --₩n" ;
    cout << matArr[i] << endl;
}
```

```
-- Matrix 0 --
|    1   2 |
|    3   4 |

-- Matrix 1 --
|    1  -1 |
|    0   2 |

-- Matrix 2 --
|    1   2   3 |
|    3   2   1 |

-- Matrix 3 --
|    2   1 |
|    3   6 |

-- Matrix 4 --
| -999 |

-- Matrix 5 --
|    1   3 |
|    3   5 |

-- Matrix 6 --
| -999 |

-- Matrix 7 --
|    7   6   5 |
|   15  14  13 |
```